Notes on Database Concepts: ACID & Normalization

Utkarsh Sahu

September 24, 2025

1 Part 1: Database Keys (The "Identifiers")

Keys are fundamental to relational databases. They are special columns used to uniquely identify records (rows) in a table and to create links between tables. Here are the main types of keys in SQL:

1.1 Primary Key

This is a column or a set of columns that uniquely identifies each row in a table. A primary key cannot contain NULL values and must be unique for each record. Each table can have only one primary key.

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    FirstName VARCHAR(50),
    LastName VARCHAR(50)
);
```

1.2 Foreign Key

A foreign key is a column or a set of columns in one table that refers to the primary key in another table. It establishes a link between two tables, enforcing referential integrity. Foreign keys can contain duplicate values and NULL values.

```
CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100),
    StudentID INT,
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);
```

1.3 Candidate Key

A candidate key is a column or a set of columns that can uniquely identify a row in a table. It is a superkey with no redundant attributes. All candidate keys are superkeys, but not all superkeys are candidate keys. A primary key is chosen from the set of candidate keys.

1.4 Super Key

A super key is a set of columns that uniquely identifies each row in a table. It can contain redundant attributes that are not strictly necessary for unique identification.

1.5 Unique Key

Similar to a primary key, a unique key ensures that all values in a column are distinct. However, unlike a primary key, a unique key can accept one NULL value.

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Email VARCHAR(100) UNIQUE,
    Name VARCHAR(50)
);
```

1.6 Composite Key

A composite key is a candidate key that consists of more than one attribute (column) to uniquely identify a row in a table.

```
CREATE TABLE OrderItems (
    OrderID INT,
    ProductID INT,
    Quantity INT,
    PRIMARY KEY (OrderID, ProductID)
);
```

---

2 Part 2: ACID Properties (The "Safety Rules" for Data)

Think of ACID properties as a set of unbreakable promises a database makes every time data is changed. It ensures that every transaction is handled safely and reliably, preventing data corruption or loss.

> **Analogy:** Imagine depositing a check at an ATM. The transaction involves two steps: the ATM accepts your check, and your account balance increases. ACID ensures this entire process is safe, even if the power goes out halfway through.

2.1 A is for Atomicity (All or Nothing)

This rule ensures that a transaction is treated as a single, indivisible unit. Either the *entire* transaction succeeds, or *none* of it does. There's no middle ground.

- **Simple Explanation:** A transaction can't be left half-done.
- **Analogy:** Booking a round-trip flight. You need both the flight *to* your destination and the flight *back*. If the return flight can't be booked for some reason, an atomic system would automatically cancel the outbound flight booking too. You either get the complete round trip or nothing.
- **Example:** When you move an item from your "Shopping Cart" to a completed "Order" on an e-commerce site, two things must happen:
  1. The item is removed from your cart.
  2. The item is added to your order history.

  Atomicity guarantees you won't have an item disappear from your cart but fail to show up in your orders.

2.2 C is for Consistency (Follow the Rules)

This ensures that any transaction will only take the database from one valid state to another. It prevents any data that violates the database's own rules from being written.

- **Simple Explanation:** The data must always make sense.
- **Analogy:** A phone's contact list. A rule might be that every contact *must* have a name. You shouldn't be able to save a phone number without a name attached to it. Consistency enforces these kinds of rules.

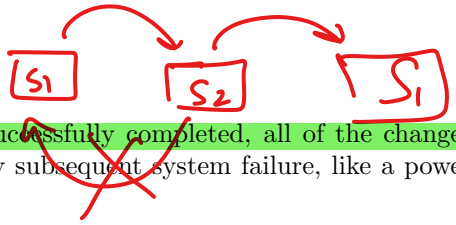
- **Example:** A company's employee database has a rule that every employee must be assigned to a valid, existing department. If you try to assign a new employee to "Department X," but "Department X" doesn't exist, the database will reject the transaction to maintain consistency.

2.3 I is for Isolation (Don't Bother Me)

Isolation ensures that transactions that are running at the same time do not interfere with each other. Each transaction feels like it's the only thing happening in the database at that moment.

- **Simple Explanation:** Concurrent transactions won't trip over each other.
- **Analogy:** Two people editing the same text document at the same time. Without isolation, one person's edits could randomly appear in the middle of the other's sentence, creating a jumbled mess. Isolation ensures that one person's changes are saved completely before the other person sees them.
- **Example:** Imagine two people trying to book the last available seat on a flight. The isolation property ensures the system processes one request completely, books the seat for that person, and then tells the second person that the seat is no longer available. It prevents the seat from being sold to both people simultaneously.

2.4 D is for Durability (It's Saved for Good)



This property guarantees that once a transaction has been successfully completed, all of the changes it made to the database are permanent. They will survive any subsequent system failure, like a power outage or crash.

- **Simple Explanation:** Once a transaction is done, it stays done.
- **Analogy:** Sending an important email. Once you hit "Send" and it confirms it was sent, you trust that the email system has saved it and it won't just disappear, even if your computer crashes a moment later. Durability is that guarantee for a database.
- **Example:** After you transfer money from your savings to your checking account, you get a confirmation. Durability ensures that this transfer is permanently recorded. Even if the bank's entire computer system goes down for maintenance right after, the record of your transfer will still be there when it comes back online.

---

3 Part 3: Normalization (The "Art of Tidying Up" Data)

Normalization is the process of organizing data in a database to reduce repetition and improve data integrity. It involves breaking down large, messy tables into smaller, more manageable ones.

> **Analogy:** Imagine you have a single, massive, messy spreadsheet to run your whole business. Normalization is like taking that spreadsheet and splitting it into clean, organized lists: one for Customers, one for Products, and one for Orders. This makes it much easier to find and update information without making mistakes.

3.1 First Normal Form (1NF): One Value Per Box

This is the most basic rule. It states that every cell in your table must hold only a single value, and each row must be unique.

- **Simple Explanation:** No stuffing multiple pieces of info into one box.
- **Analogy:** A recipe card. Instead of having an `Ingredients` section that reads "1 cup flour, 2 eggs, 1 tsp vanilla", 1NF would have a separate line for each ingredient.

Before 1NF:

Order ID	Customer	Items Ordered
101	John Smith	Pizza, Soda, Wings

After 1NF:

Order ID	Customer	Item Ordered
101	John Smith	Pizza
101	John Smith	Soda
101	John Smith	Wings

3.2 Second Normal Form (2NF): Relate to the Whole Thing

This rule applies when a table's unique identifier is made of multiple columns. It says that all other columns must depend on the *entire* identifier, not just a part of it.

- **Simple Explanation:** All information in a row must be about the full unique ID.

Before 2NF:

Student_ID	Class_ID	Student_Major	Grade
S1	C101	Physics	A
S1	C202	Physics	B

(Problem: The major "Physics" is repeated and only relates to the student, not the class.)

After 2NF (Split into two tables):

Students Table:

Student_ID	Student_Major
S1	Physics

Enrollment Table:

Student_ID	Class_ID	Grade
S1	C101	A
S1	C202	B

3.3 Third Normal Form (3NF): No Chain Reactions

This rule says that you should remove any columns that don't depend directly on the primary unique ID, but rather on another column in the table.

- **Simple Explanation:** Avoid storing information that you can find somewhere else.

Before 3NF:

Employee_ID	Employee_Name	Department	Dept_Location
E1	Alice	Sales	New York
E2	Bob	Marketing	Chicago
E3	Charlie	Sales	New York

(Problem: `Dept_Location` depends on `Department`, not directly on the `Employee_ID`.)

After 3NF (Split into two tables):

Employees Table:

Employee_ID	Employee_Name	Department
E1	Alice	Sales
E2	Bob	Marketing
E3	Charlie	Sales

Departments Table:

Department	Dept_Location
Sales	New York
Marketing	Chicago

3.4 Boyce-Codd Normal Form (BCNF): No Hidden Keys

BCNF is essentially a stricter, more thorough version of 3NF. It handles very specific types of redundancy that 3NF might miss.

- **Simple Explanation:** BCNF says that if any column or group of columns can determine the value of another column, then that first group of columns must be a "candidate key"—meaning, it could have been the primary unique ID for the whole table.

Before BCNF:

Player	Day	Coach	Coach_Specialty
Sam	Monday	Jones	Serving
Sam	Wednesday	Davis	Forehand
Alex	Monday	Jones	Serving

(Problem: 'Coach' determines 'Coach_Specialty', but 'Coach' is not the primary key.)

After BCNF (Split into two tables):

Session Table:

Player	Day	Coach
Sam	Monday	Jones
Sam	Wednesday	Davis
Alex	Monday	Jones

Coach Specialty Table:

Coach	Specialty
Jones	Serving
Davis	Forehand

(Note: There are higher levels of normalization, but these first three solve the vast majority of data organization problems.)